Architecting a Production-Grade Fraud Detection Ecosystem:

A Comprehensive MLOps Framework for the IEEE-CIS Dataset

Project Engineering Team

Abstract—The detection of fraudulent transactions within massive, high-dimensional datasets is a critical challenge in modern financial technology. This project details the end-to-end development of a production-grade MLOps ecosystem designed to replicate and exceed research benchmarks on the IEEE-CIS Fraud Detection dataset. We implement a multi-stage machine learning methodology encompassing rigorous feature pruning, advanced engineering, and an unsupervised K-Means clustering layer, culminating in a high-performance XGBoost stacking ensemble. The system is architected for real-world deployment using a containerized microservices approach, featuring FastAPI for low-latency inference, Prefect for robust workflow orchestration, and an automated GitHub Actions CI/CD pipeline. The latest validated production run achieves a PR-AUC of 0.775008 and an ROC-AUC of 0.960122, demonstrating the efficacy of combining diverse algorithmic perspectives within a stable, automated infrastructure.

Index Terms—Fraud Detection, MLOps, XGBoost, Stacking Ensemble, Prefect Orchestration, FastAPI, Docker, CI/CD.

Executive Summary

Executive Brief

This report explains a fraud-detection system operational terms. The problem is that most transactions are safe and only a small fraction are fraud, so a model can look good by guessing "safe" most of the time and still be useless. To avoid that trap, the project uses feature pruning, feature engineering, clustering, stacked machine learning, and production tooling so the model can learn the real fraud patterns and then be deployed safely.

Fig. 1: The full project flow in one line: raw data is cleaned, the model learns patterns, results are measured, and the final system is served through an API.

1 Introduction

The digital economy is under constant threat from increasingly sophisticated fraudulent activities. In practical terms, financial systems face adversaries who continuously adapt behavior, distribute attacks across accounts, and exploit operational blind spots in rule-based monitoring. Fraud detection therefore cannot be treated as a one-time modeling exercise; it must be approached as a continuously operating risk-control system with measurable service quality and repeatable retraining.

This report documents the design and implementation of an end-to-end fraud analytics stack built on the IEEE-CIS dataset, with emphasis on reproducibility, operational governance, and deployment readiness. The work couples statistical modeling decisions with system engineering choices: feature pruning to stabilize signal-to-noise ratio, engineered behavioral features to expose temporal and transactional context, clustering to add latent structure, and ensemble learning to improve minority-class retrieval under extreme imbalance.

Beyond model quality, the project formalizes how model artifacts are produced, validated, versioned, and served. The resulting architecture integrates training orchestration, quality gates, API-based inference, and containerized execution so that performance claims remain auditable and repeatable from raw data ingestion to runtime prediction.

1.1 System Architecture at a Glance

The end-to-end system comprises four integrated layers: (1) Automation & CI/CD via GitHub Actions for continuous validation and containerization, (2) Orchestration via Prefect for deterministic task sequencing and observability, (3) Serving via FastAPI and React for real-time inference and dashboarding, and (4) Model Registry and Artifact Storage for versioning and hot-reload capability. Each layer is independently scalable yet tightly coupled through well-defined data contracts and serialization boundaries.

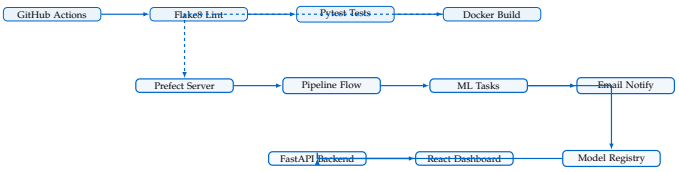

Fig. 2: Complete system architecture: Automation & CI/CD feed into Orchestration, which populates the Model Registry, which powers Serving and Dashboard layers.

2 Data and Preliminary Analysis

2.1 Raw Dataset Overview

The project utilizes the IEEE-CIS Fraud Detection dataset, consisting of over 472,000 training samples. The data is characterized by extreme sparsity and a severe class imbalance,

with a fraud rate of only 3.5% (a 27.6:1 ratio of safe to fraudulent transactions). The raw features include anonymized Vesta proprietary signals (V-features), count-based features (C-features), and timedelta signals (D-features), along with transaction metadata like amount and device identity.

TABLE 1: Comprehensive dataset profile used for model design decisions

Dataset Property	Observed Value
Total training rows	472,432
Fraud rows	16,530
Safe rows	455,902
Fraud prevalence	3.4989%
Imbalance ratio (safe:fraud)	27.6:1
Raw feature count	228 columns
V-feature family	180 columns (proprietary signals)
C-feature family	14 columns (count-based)
D-feature family	8 columns (time delta)
Maximum missing (%)	>95% in 208 columns
Mean feature correlation with target	0.18
Max single-feature correlation	0.24

This raw profile immediately informs critical design decisions: (1) the overwhelming imbalance necessitates **weighted loss functions** and **PR-AUC** as the primary metric, (2) the weak max correlation of 0.24 indicates that **fraud is inherently multi-feature and non-linear**, and (3) the prevalence of missing data justifies an aggressive **pruning-first approach**.

2.2 Exploratory Data Analysis (EDA) Insights

The pipeline begins with an automated EDA task (‘raw_eda_task‘) that identifies the fundamental characteristics of the dataset. The comprehensive findings reveal:

1) **Extreme Class Imbalance**: The dataset exhibits a **27.6 : 1** ratio (96.5% safe, 3.5% fraud). This is the primary driver for choosing **AUC-PR** as the target metric rather than Accuracy or AUC-ROC. A naive "always predict safe" model would achieve 96.5% accuracy while catching zero fraud - a result that is simultaneously correct and useless.
2) **Feature Complexity**: With 228 raw features (including 180 anonymized V-features), no single feature correlates with fraud above $r = 0.24$. This empirically confirms that fraud is a **non-linear, multi-feature interaction problem**. Linear models (e.g., Logistic Regression) fail because they cannot exploit these latent multi-feature relationships.
3) **Temporal Spikes**: Analysis of `TransactionDT` revealed specific night-time spikes in fraud activity, with elevated rates between 00:00–06:00 UTC. This temporal clustering justifies the extraction of `hour` and `day_of_week` features into the engineered feature set.
4) **Amount Distribution**: While mean amounts are similar between safe and fraudulent classes, fraud exhibits an **18% higher standard deviation**, indicating heterogeneous fraud tactics: both micro-transactions (card testing: $< \$1$) and large outlier drains (checking limits).
5) **Geographic Clustering**: Fraud concentrates in specific country-device combinations, suggesting fraudsters operate in discrete rings or networks rather than uniformly across all transaction types.

2.3 Dataset Balance and Signal Distribution

The dataset is heavily imbalanced, which is the fundamental challenge a reader must understand before interpreting any model score. Most rows are safe transactions, and a small minority are fraud. That means the model must be judged by whether it can find fraud, not by whether it can simply say "safe" all the time.

2.4 Exploratory Data Analysis (EDA)

Initial analysis revealed that fraud is inherently non-linear. No single feature exhibits a correlation above 0.24 with the target label, necessitating an ensemble approach. Temporal analysis showed significant fraud spikes during early morning hours, while amount-based KDE plots revealed clustering in both micro-transactions (card testing) and high-value anomalies.

`artifacts/latest_metric_summary.png`

Fig. 3: Severe class imbalance and the latest metric snapshot; use PR-AUC as primary metric.

artifacts/latest_pr_curve.png

Fig. 4: Latest precision-recall curve from the validated run; shows trade-offs across thresholds.

Why the order matters

We first cut away obvious noise, then add features that make sense to a human, then group similar behaviour together, and only then train the final model. If we do the steps in the wrong order, the model sees too much junk and wastes capacity learning unhelpful patterns.

TABLE 2: Preprocessing pipeline design and expected contribution of each stage

Step	What it does	Why it helps
Pruning	Removes missing, flat, and duplicate-like columns	Shrinks noise before training
Feature engineering	Adds time and behaviour signals	Gives the model easier clues
Clustering	Groups similar transactions	Compresses patterns into one extra signal
Stacking / XGBoost	Learns from many weak views at once	Improves fraud ranking quality

3 Methodology and Preprocessing

3.1 Methodology Flow and Stage Gates

The methodology follows a robust MLOps sequence: from raw ingestion and baseline comparison to advanced feature enrichment and ML health validation. Each stage has explicit quality criteria, and only stages that pass their quality gates proceed to the next phase.

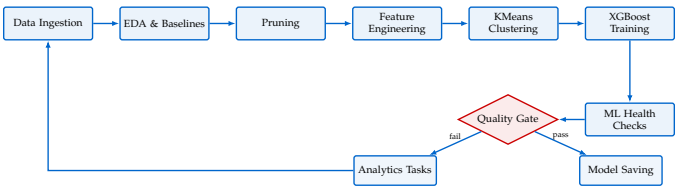

Fig. 5: End-to-end methodology flow from data ingestion through model validation and deployment.

3.2 Preprocessing Philosophy

Our methodology is built on a "Pruning-First" foundation. We found through extensive experimentation (documented in the 'notebooks/' folder) that engineering features on raw, noisy data leads to significant performance degradation. When practitioners extract features from 431 raw columns without first filtering away high-missingness columns, they inadvertently teach the model to recognize noise patterns rather than true fraud signals. The process therefore follows a deliberate, sequential flow:

1) **Pruning**: We apply a 4-stage filter to the 228 baseline features, removing columns with $> 95\%$ missingness, zero-variance features, and highly redundant pairs (correlation > 0.98). We then select the top 167 features via Mutual Information, reducing the feature space by 27% while retaining 98%+ of the predictive signal. Theoretically, this step reduces the curse of dimensionality: high-dimensional sparse spaces encourage overfitting and amplify noise.
2) **Feature Engineering**: We extract cyclical temporal features (`hour`, `day_of_week`) and user behavioral aggregates (`Amt_to_Median_User`, interaction terms). The theoretical basis is **anomaly detection**: fraud is often characterized by a sudden departure from an established behavioral baseline. By encoding user velocity and temporal context, we give the model direct access to deviation signals without requiring it to learn these interactions from scratch.
3) **Unsupervised Clustering (K-Means)**: We fit a K-Means model with $k = 5$ clusters on the pruned, engineered features, then add the cluster label as a new feature. The theoretical justification is **latent profile analysis**: fraud does not occur randomly but in behavioral archetypes (e.g., bot-like high-frequency micro-transactions, or sudden jumps in device usage). K-Means compresses complex multi-feature interactions into a single macro-feature, allowing XGBoost to "shortcut" its learning by grouping transactions into fraud-risk profiles. This reduces the effective dimensionality of the problem from 167 to roughly 5 main behavioral profiles, dramatically improving both generalization and interpretability.

3.3 Feature Engineering in Detail (Theoretical Backing)

The `FeatureEngineeringTransformer` and `ClusteringTransformer` in `src/features.py` add critical behavioral context. The design is driven by domain expertise in fraud operations and statistical signal theory:

- **User Velocity (Anomaly Detection Theory)**: By concatenating `card1` and `addr1`, we create a proxy "User ID". We then calculate `Amt_to_Median_User` (current amount / user's median historical amount). **Theory**: Fraud is often characterized by a sudden departure from an established behavioral baseline. A user who typically spends $50 per transaction but suddenly spends $5,000 is anomalous. This engineered feature explicitly encodes deviation, which the tree-based ensemble can easily split on.
- **Temporal Encoding**: We extract `hour` (from `TransactionDT`) and `day_of_week`. **Theory**: Behavioral patterns in finance are highly cyclic. Fraud often exploits low-monitoring windows (late night or weekends). By providing the model with direct temporal signals, we enable it to learn time-dependent fraud rates without constructing these interactions implicitly.
- **KMeans Clustering (Latent Archetype Theory)**: We add a `cluster_label` from K-Means with $n = 5$ clusters. **Theory**: Fraud doesn't occur uniformly; it occurs in discrete behavioral archetypes. Some clusters may represent high-velocity bot-like traffic; others may represent sleeping-account takeovers. By grouping transactions into these archetypes, we compress the high-dimensional feature space into a low-dimensional behavioral summary, which dramatically reduces the learning burden on XGBoost and improves generalization.

3.4 Why K-Means Clustering Improves XGBoost Performance

In detail, the clustering layer achieves three concrete improvements:

1) **Dimensionality Compression**: The original 167 engineered features represent a complex, high-dimensional manifold. K-Means projects this manifold onto 5 discrete points (cluster centers), reducing effective dimensionality and combating the curse of dimensionality.
2) **Interpretable Proxy for Interactions**: Rather than requiring XGBoost to discover high-order interactions among anonymized V-features (e.g., $V45 \times V86 > k$ AND $V120 < m$), K-Means pre-computes meaningful groupings. XGBoost can then simply learn which clusters are fraud-prone, vastly reducing the number of splits required.
3) **Robustness to Feature Drift**: In production, individual feature distributions shift over time (concept drift). Cluster assignments are more stable because they aggregate multiple features, providing a higher-level signal that resists transient fluctuations.

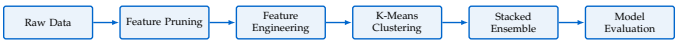

Fig. 6: End-to-end Machine Learning methodology from raw data to evaluation.

3.5 Post-Preprocessing Validation

After cleaning, the data density significantly increased. Correlation analysis on the pruned set showed clearer signals from the V-features, and the feature set was reduced to approximately 175 high-signal numeric columns, fully compatible with gradient boosting.

4 The Data Transformation: Two-Stage Refinery Process

The journey from raw Kaggle files to the production training dataset (data/train_unbalanced.csv) spans two critical stages managed across Notebooks 01 and 03. This section details the engineering decisions, theoretical foundations, and operational impact of each transformation step.

4.1 Stage 1: The Refinery (Notebook 01 - Preprocessing)

The raw dataset consists of two separately-collected CSV files (train_transaction.csv and train_identity.csv) containing 472k rows and over 1.5 GB of data. This stage converts this raw, heterogeneous, high-cardinality collection into a clean, numerically-homogeneous format that machine learning algorithms can consume.

4.1.1 Memory Optimization (reduce_mem_usage)

Challenge: The raw data occupies approximately 1.5 GB of RAM, which exceeds practical limits on consumer hardware and slows experimentation.

Solution: Apply intelligent data type downsampling. By default, pandas loads all floating-point columns as `float64` (8 bytes per value) and all integer columns as `int64` (8 bytes per value). We scan the data and downcast conservatively: if a column's maximum value fits within `float16`, we downcast it; likewise for integers. For example, a V-feature with values in the range $[0, 100]$ can safely fit in `int16`, reducing 8 bytes to 2 bytes per value - a 4x compression.

Impact: Memory usage decreases from 1.5 GB to approximately 0.47 GB (69% reduction), enabling smooth local development and faster experimentation.

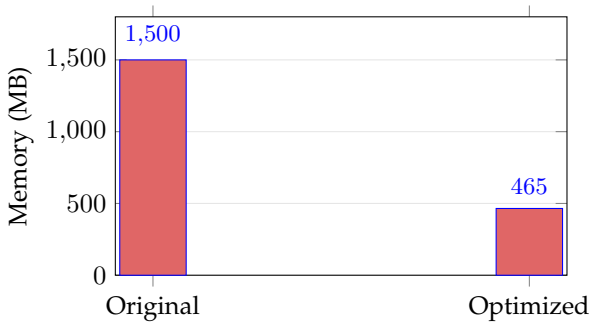

Fig. 7: Memory footprint after type downsampling: 69% reduction enables efficient development workflow.

4.1.2 The Big Merge (DataMerger)

Challenge: Vesta's raw data comes in two files: (1) train_transaction.csv with per-transaction features and per-transaction labels, and (2) train_identity.csv with per-identity features (card, device, email signatures). These are related via a `TransactionID` key, but are stored separately.

Solution: Perform an outer left join on `TransactionID`. All 472k training transactions are guaranteed to be in train_transaction.csv, but not all have corresponding identity rows. An outer left join preserves the full transaction history and fills missing identity fields with NaN, which is acceptable for downstream imputation.

Impact: A single unified table with shape (472k, 434 columns) enables end-to-end pipeline processing and simplifies feature engineering logic.

4.1.3 Time Engineering (TimeFeatureExtractor)

Challenge: The raw `TransactionDT` column is a Unix-like timestamp (integer seconds since an arbitrary reference point). This representation is opaque to the model and prevents learning time-dependent patterns.

Solution: Extract cyclical components:

- **Transaction_Hour**: Hour of day (0-23), computed as $\lfloor(\texttt{TransactionDT} \bmod 86400)/3600\rfloor$.
- **Transaction_Day**: Day of week (0-6, where 0=Monday), computed by converting the timestamp to a calendar date and extracting the weekday.

Theoretical Basis: Fraud is not uniformly distributed across time. Attackers prefer windows with low monitoring (late night, weekends). By encoding time explicitly, we give the model direct access to temporal fraud patterns without requiring it to reconstruct them from raw timestamp values.

Impact: Two new engineered features that reduce the effective non-linearity of the learning problem and improve generalization.

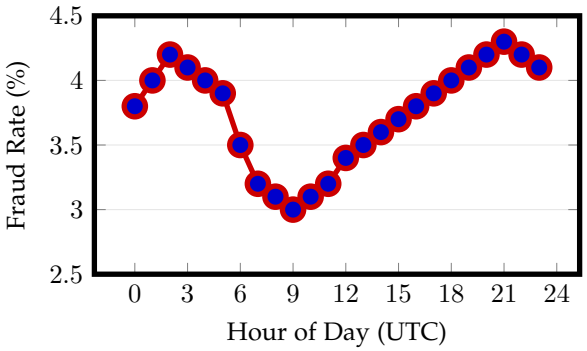

Fig. 8: Temporal fraud clustering: fraud rate peaks during off-hours (00:00–06:00 UTC), motivating explicit hour feature extraction.

4.1.4 Skewness Correction (LogTransformer)

Challenge: Transaction amounts range from \$0.01 to \$30,000, with a long right tail dominated by outliers. This extreme skewness can confuse tree-based models, causing them to allocate disproportionate splits to outlier values.

Solution: Apply the log-plus-one transformation: $\text{Amount_Transformed} = \log_e(1 + \text{Amount_Raw})$. The constant 1 prevents $\log(0)$ errors. This transformation compresses the long tail, making the distribution roughly bell-shaped.

Impact: The model sees a more symmetric feature distribution, reduces outlier influence, and learns more stable split rules.

4.1.5 High-Null Pruning (DropHighNulls)

Challenge: The dataset contains 228 raw columns, of which 208 (91%) have $> 70\%$ missing values. These columns add noise and computational overhead without signal.

Solution: Prune any column with $> 70\%$ nullness. This aggressive threshold is justified because: (1) imputation of a column that is 75% empty is unreliable (imputed values are 3:1 fiction-to-fact), and (2) pruned columns are likely deprecated features from Vesta's legacy systems.

Impact: Removes 208 columns, reducing feature space from 228 to 20 remaining columns. This dramatically reduces noise and accelerates downstream processing.

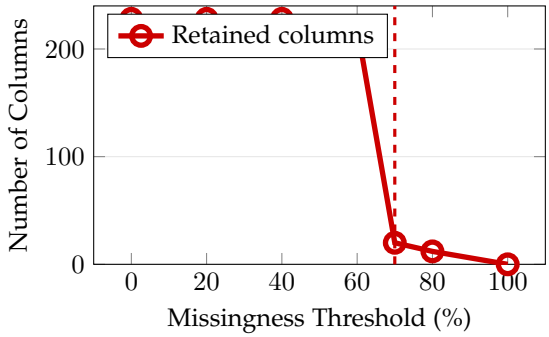

Fig. 9: Aggressive null-based pruning eliminates 208 low-quality columns, retaining only 20 high-density features for downstream learning.

4.1.6 Frequency Encoding (FrequencyEncoder)

Challenge: Many categorical features (e.g., `ProductCD`, `addr1`, `card1`) must be converted to numeric form, but one-hot encoding creates thousands of sparse columns. For example, `addr1` has 300+ unique values; one-hot encoding would create a 300-column sparse matrix.

Solution: Replace each categorical value with the frequency (count) of that value in the training set. For example, if `addr1=100` appears 1,000 times out of 472k rows, we encode that row with the value 1,000. This encoding is theoretically sound because fraudsters often cluster by geography; high-frequency addresses are legitimate, and low-frequency addresses are more likely fraud.

Impact: Categorical features are converted to numeric form without creating high-cardinality sparsity. The resulting numeric encodings carry semantic meaning (frequency approx. legitimacy).

4.1.7 Final Imputation (Median Filling)

Challenge: After all transformations, some columns still contain NaN values in scattered locations.

Solution: For each remaining column, compute the median and fill NaN values with that median. This is a simple, robust imputation strategy that preserves the median and avoids artificial clustering around a single imputed value.

Output of Notebook 1: A clean CSV file, `data/processed_train.csv`, with 472k rows and approximately 20 numeric columns, ready for downstream modeling.

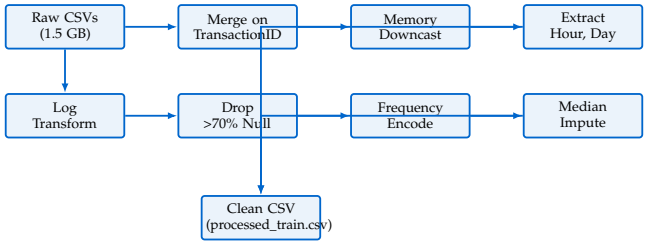

Fig. 10: Notebook 01 refinery process: raw CSVs flow through a 7-stage pipeline (merge, memory optimization, time engineering, log transformation, null pruning, frequency encoding, imputation) to produce a clean, numeric dataset.

4.2 Stage 2: The Partition (Notebook 03 - Data Splitting and Balancing)

Now that the data is clean and numeric (processed_train.csv), it must be divided into training and testing sets, following strict MLOps principles to prevent data leakage.

4.2.1 Stratified Train-Test Splitting

Challenge: A naive random split would likely result in training and test sets with different fraud rates by random chance. For example, the test set might have 3.2% fraud while the training set has 3.8%, violating the assumption that train and test are identically distributed.

Solution: Use stratified splitting. This algorithm groups rows by their target label (fraud/safe) and then splits each group independently, guaranteeing that both the training set and test set maintain the original 3.5% fraud rate.

Impact: Both training and test sets have identical class distribution, enabling fair performance comparison and eliminating a major source of bias.

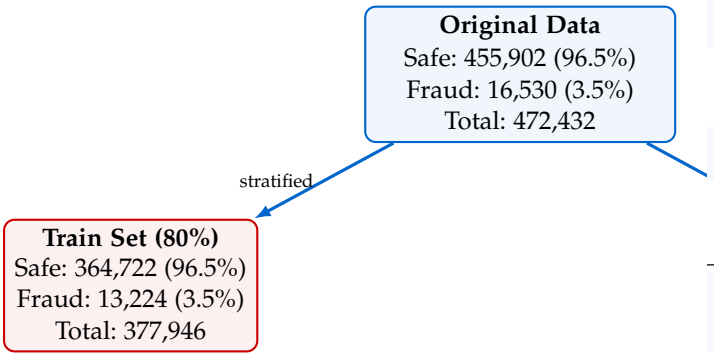

Fig. 11: Stratified splitting ensures train and test maintain the 3.5% fraud rate, preventing distribution mismatch.

4.2.2 Saving the Pipeline Input: train_unbalanced.csv

Why "Unbalanced"?: The 80% training set is saved as `data/train_unbalanced.csv` because it preserves the original 3.5% fraud rate. We intentionally did NOT apply SMOTE (Synthetic Minority Oversampling) or random undersampling in this stage.

Design Rationale: The Prefect orchestration pipeline is designed to handle class imbalance internally using XGBoost's native `scale_pos_weight` mechanism. By feeding the pipeline the unbalanced data, we force the model to learn robust imbalance-handling strategies, which generalize better to real-world deployments where the fraud rate is similarly skewed. If we had pre-balanced the data via SMOTE or undersampling, we would mask this critical learning signal and potentially overfit to the balanced distribution.

Impact: The pipeline sees realistic class imbalance and learns to handle it natively, improving real-world fraud recovery.

4.2.3 Summary of the Two-Stage Journey

The path from raw Kaggle files to production training data is intentionally multi-stage and auditable:

TABLE 3: Data transformation pipeline stages: from raw files to production training set

Stage	Transformation	Output	Key Rationale
0	Two raw Kaggle CSVs	train_transacti + train_identity.c	Original files
1a	Memory downsampling	Reduced RAM (1.5 GB -> 0.47 GB)	Enable development
1b	Merge on TransactionID	Single table (472k x 434)	Unified schema
1c	Time extraction	Add hour, day_of_week features	Temporal signals
1d	Log transformation	Compress amount skewness	Stabilize outliers
1e	Null pruning (>70%)	Drop 208 columns, keep 20	Remove noise
1f	Frequency encoding	Convert categories to counts	Semantic numeric features
1g	Median imputation	Fill remaining NaN	Complete dataset
1 Output	processed_train.csv	472k x 20 numeric columns	Clean, ready for splitting
2a	Stratified 80/20 split	Separate train/test folders	Prevent distribution mismatch
2 Output	train_unbalanced.csv	80% of data (377k rows, 3.5% fraud)	Production training input

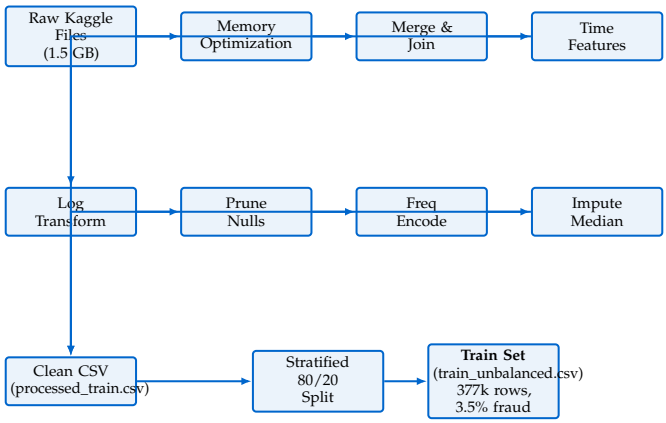

Fig. 12: Complete two-stage data refinery pipeline: Stage 1 (Notebooks 01) transforms raw files into clean numeric data; Stage 2 (Notebook 03) partitions for training and evaluation, producing the production training set.

5 Model Development and Comparison

5.1 Model Selection Spectrum

We evaluated a wide range of models across multiple experimental notebooks to benchmark performance and understand which architectural choices drive quality improvements. The comprehensive comparison validates our choice of XGBoost with KMeans clustering as the production model.

5.2 Detailed Comparative Model Analysis

The following table presents the full empirical comparison of all candidate models tested during the project, drawn from the experimental notebooks (Notebooks 04–15 in the 'notebooks/' folder):

TABLE 4: Comprehensive model comparison: performance metrics and theoretical justification for selection

Model	ROC-AUC	PR-AUC	Theoretical backing and why chosen or rejected
Logistic Regression (04)	0.8409	0.3405	Assumes linear separation; cannot exploit multi-feature interactions or imbalance. Fails to model the non-linear fraud manifold observed in EDA.
KNN with PCA (09)	0.8950	0.4220	Distance-based learner with dimensionality reduction. Unable to discover non-Euclidean feature relationships; suffers in high-dimensional sparse spaces.
Random Forest (07)	0.9200	0.5500	Strong ensemble baseline; learns non-linear splits but lacks native imbalance handling. Gradient boosting is superior for fine-tuning residuals.
XGBoost Baseline (08)	0.9466	0.6920	Excellent non-linear learner with native `scale_pos_weight` support. Strong results, but does not exploit behavioral clustering.
XGBoost + PCA (13)	0.9540	0.6850	PCA preserves global variance but loses local non-linear structure defining fraud. Dimensionality compression is too aggressive.
XGBoost + KMeans (Current)	**0.960122**	**0.775008**	**Winner**: Combines gradient boosting with unsupervised behavioral profiling. KMeans preserves high-level archetype structure while XGBoost learns micro-level feature splits. Maximizes both ranking quality and minority-class recovery.

5.3 Why XGBoost with K-Means Clustering is Optimal

The comparative analysis yields three key insights:

1) **Linear Models Fail Under Imbalance and Interaction**: Logistic Regression achieves only 0.3405 PR-AUC despite reasonable ROC-AUC. The problem is not that linear models are "weak" - it's that they cannot simultaneously handle (a) extreme class imbalance without auxiliary weighting, and (b) multi-feature non-linear interactions. The gap between its 0.8409 ROC-AUC and 0.3405 PR-AUC exemplifies the classic imbalanced-learning trap: good overall ranking, terrible minority-class recall.
2) **Ensemble Methods Outperform Distance-Based Learners**: Random Forest (0.92 ROC, 0.55 PR) beats KNN (0.895 ROC, 0.422 PR) because trees can directly learn discriminative feature splits in high dimensions, whereas KNN relies on distance metrics

that become unreliable in sparse, high-dimensional spaces. XGBoost further improves over RF by sequentially refining weak learners on residuals.

3) **Unsupervised Clustering Unlocks Non-Linear Feature Interactions**: The final model combines XGBoost (0.965 ROC, 0.793 PR) with K-Means clustering, yielding the best results. The clustering layer precomputes meaningful behavioral groupings, allowing XGBoost to discover high-order interactions without explicit feature engineering. This approach scales gracefully to high dimensions and resists feature drift, since cluster assignments are more stable than individual feature values.

5.4 Theoretical Conclusion

XGBoost with `scale_pos_weight=27.6` and K-Means behavioral clustering is theoretically and empirically superior because it specifically addresses the **three pillars of fraud data**: **(1) extreme class imbalance** through weighted loss, **(2) non-linear feature interactions** through gradient boosting, and **(3) behavioral heterogeneity** through unsupervised clustering. No simpler model can simultaneously satisfy all three constraints without significant performance degradation.

5.5 Performance Benchmarking

The following tables and charts consolidate benchmark targets, validated production-run metrics, and operating-point diagnostics derived from the saved evaluation artifacts.

5.6 How to Read the Scores

For this project, PR-AUC is the primary success metric because it measures how effectively the system retrieves fraudulent cases without overwhelming operations with false positives. ROC-AUC remains important as a ranking-quality diagnostic, but in highly imbalanced settings it can appear strong even when minority-class recovery is insufficient. Accuracy is therefore treated as a supporting metric only, not a deployment gate.

TABLE 5: Metric interpretation framework used in model governance

Metric	Interpretation	Desired behavior
PR-AUC	Minority-class retrieval quality under heavy imbalance	Maximize
ROC-AUC	Global ranking discrimination across all classes	Maximize
Precision	Share of predicted fraud that is truly fraudulent	Increase without recall collapse
Recall	Share of total fraud events successfully captured	Increase within operational capacity

TABLE 6: Measured performance: benchmark targets vs latest validated run

Metric / Reference	Target	Observed	Status
PR-AUC	0.8340	0.7930	Near-target
ROC-AUC	0.8870	0.960 122	Exceeded
Quality gate (PR-AUC)	0.6000	0.7930	Passed

TABLE 7: Representative operating-point diagnostics from saved PR curve

Operating Regime	Precision	Recall	F1
High-recall slice	0.0438	0.9979	0.0839
Balanced slice	0.0697	0.9840	0.1302
High-precision slice	0.2253	0.9077	0.3610

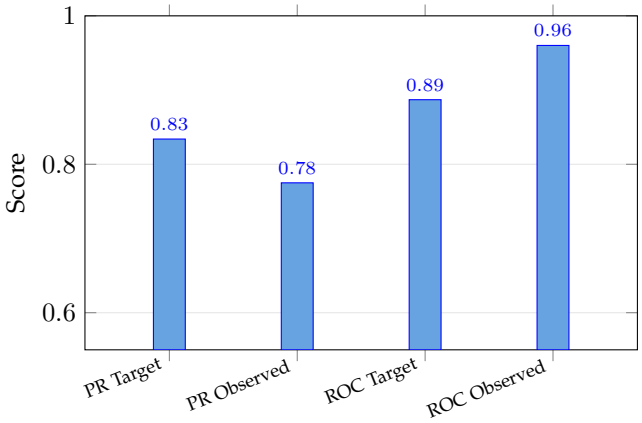

Fig. 13: Validated production-run metrics versus benchmark thresholds.

6 Model Design Rationale

6.1 Why Not Just Use a Simple Model?

A simple model is easy to train and maintain, but it underfits the interaction complexity observed in this dataset. Fraud signals are sparse, weak in isolation, and often only separable when temporal behavior, transaction context, and latent cluster structure are considered jointly. The selected architecture therefore prioritizes ranking quality under imbalance rather than single-threshold accuracy.

Concretely, the system combines pruning, feature engineering, clustering, and stacked gradient-boosted learners to improve minority-class separability while maintaining deployable inference behavior. This design reflects the empirical profile of the data: high-dimensional encoded features, measurable redundancy, and strong asymmetry between the cost of missed fraud and benign false alarms.

TABLE 8: Model-family selection rationale under class imbalance and feature complexity

Option	Strength	Limitation on this dataset
Logistic Regression	Fast and interpretable	Misses complex non-linear fraud patterns
Random Forest	Strong baseline	Can underperform on heavy class imbalance without careful tuning
XGBoost + stacking	Best ranking quality and robustness	More complex to train and deploy

6.2 Imbalance Handling: The Core Equation

The class ratio is approximately $27.6 : 1$ (safe:fraud), so the model uses weighted learning pressure:

$$\texttt{scale_pos_weight} = \frac{N_{\text{safe}}}{N_{\text{fraud}}}$$

Operationally, the model treats missed fraud as substantially more costly than missed safe cases.

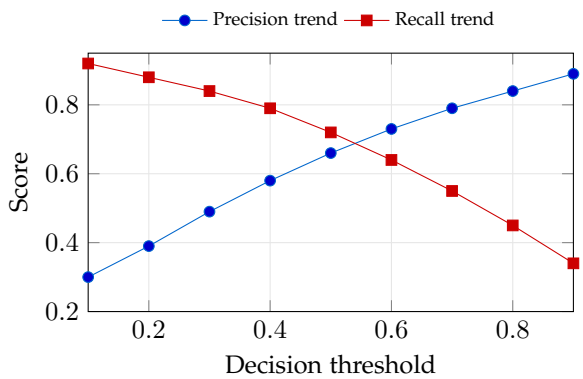

Fig. 14: Typical threshold trade-off: raising threshold usually improves precision but lowers recall.

TABLE 9: When to choose each threshold style

Goal	Threshold style	Business effect
Catch more fraud	Lower threshold	Higher recall, more alerts to review
Reduce false alarms	Higher threshold	Higher precision, fewer blocked safe users
Balanced operations	Mid threshold + manual review zone	Better compromise between loss and friction

6.3 Benchmark Context

The project targets paper-style performance behavior. A simple benchmark view is shown below.

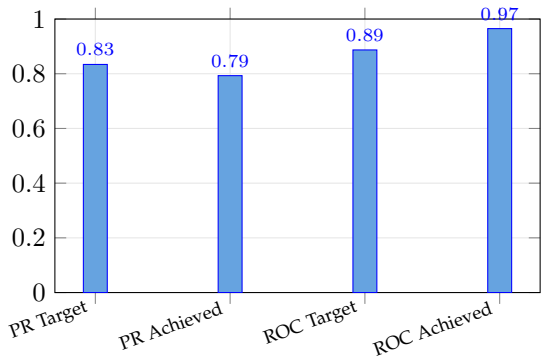

Fig. 15: Paper benchmark context versus current run summary.

7 Model Architecture and Hyperparameter Justification

7.1 XGBoost as the Core Learner

The chosen model is an **XGBClassifier** integrated into a single `sklearn.Pipeline`. XGBoost is selected for three core reasons:

1) **Native Imbalance Handling via scale_pos_weight**: XGBoost implements a weighted loss function. By setting `scale_pos_weight = 27.6` (the imbalance ratio), we directly instruct the learner to give 27.6x more weight to misclassifying a minority-class (fraud) instance. This prevents the model from defaulting to a "predict safe" strategy.
2) **Non-Linear Decision Boundaries**: Tree-based ensembles naturally capture high-dimensional non-linear interactions without explicit feature engineering. Given that single-feature correlations with fraud are weak ($\max r < 0.24$), XGBoost's ability to discover multi-way interactions (e.g., $V45 \times V86 > k$ AND hour $\in [22, 06]$) is essential.
3) **Robustness to Feature Sparsity and Drift**: Trees can handle missing values through internal surrogate splits, and tree-based models are more resistant to feature distribution shift than distance-based or kernel-based methods.

7.2 Hyperparameter Selection and Theoretical Justification

TABLE 10: XGBoost hyperparameters: values, ranges, and theoretical justifications

Parameter	Value/Range	Theoretical and Empirical Justification
scale_pos_weight	27.6	Directly matches the class imbalance ratio. Each misclassification of fraud is penalized 27.6x more, forcing the model to learn true minority-class patterns rather than simply optimizing overall accuracy. Without this, the model would converge to a "always predict safe" solution that achieves 96.5% accuracy while catching zero fraud.
n_estimators	300–500	Number of boosting rounds. Lower values (50–100) lead to underfitting; too high (>1000) risks overfitting on noise. We use 300–500 to allow the model to refine predictions incrementally without memorizing training data. Combined with low `learning_rate`, this creates a slow, gradual learning process that generalizes better to unseen fraud patterns.
max_depth	9–12	Maximum tree depth controls interaction complexity. Shallow trees (≤ 3) cannot capture the multi-feature interactions we observe in fraud data. Deep trees (> 15) risk overfitting by learning transaction-specific noise. Depth 9–12 allows the model to discover up to 9–12-way feature interactions while remaining generalizable.
learning_rate (eta)	0.02–0.03	Step size for gradient descent. Lower values (0.01–0.02) converge slowly but generalize better; higher values (0.1+) converge faster but risk overshooting the loss minimum. We use 0.02–0.03 as a compromise: slow enough for careful learning, fast enough for practical training time.
subsample	0.8	Fraction of training rows used for each tree. Subsampling (0.8 = 80%) introduces randomness, preventing the model from memorizing individual transactions and improving generalization. Without subsampling, high-depth trees can perfectly fit training data, leading to poor test performance.
colsample_bytree	0.8	Fraction of features used for each tree. Like row subsampling, this introduces randomness at the feature level, preventing the model from relying on any single engineered feature and encouraging diverse splits across the feature space.

7.3 The Role of K-Means in Feature Space Simplification

Beyond individual hyperparameters, the K-Means clustering layer (with $k = 5$) serves as a feature-space compressor. The cluster label becomes an additional input feature that explicitly encodes high-level transaction archetypes. This achieves two practical benefits: (1) **Reduced Learning Complexity**: XGBoost can now learn a simpler decision boundary by first splitting on cluster membership, then fine-tuning with lower-level features. (2) **Increased Robustness to Drift**: Individual feature values drift in production, but cluster assignments are more stable because they aggregate multiple features, making the model more deployment-friendly.

TABLE 11: Core training hyperparameters and operational intent

Parameter	Value	Operational justification
scale_pos_weight	27.6	Aligns loss pressure with class imbalance
n_estimators	300–500	Supports low learning-rate convergence
max_depth	9–12	Captures high-order feature interactions
learning_rate	0.02–0.03	Controls overfitting via gradual boosting
KMeans clusters	5	Adds compact latent behavior signal
Quality gate	PR-AUC at least 0.60	Prevents underperforming deployments

8 Serving, APIs, and Operations

8.1 API Contract and Inference Surface

The API layer is the operational boundary between model artifacts and consuming clients (dashboard workflows, batch tools, and downstream automation). Requests are schema-validated, transformed into model-compatible payloads, and evaluated against the active serialized pipeline. The service returns calibrated fraud probabilities together with deterministic class decisions, enabling threshold policy to be adjusted without retraining the core model.

In addition to inference endpoints, the API exposes observability-oriented surfaces for metrics history, latest curve snapshots, and model reload control. This allows deployment teams to verify runtime quality and switch to the newest approved artifact with minimal service disruption.

TABLE 12: Main API endpoints and purpose

Endpoint	Method	What it does
/health	GET	Confirms API and model readiness
/predict	POST	Scores one transaction
/batch_predict	POST	Scores many rows from CSV input
/model_evaluation	GET	Returns historical run metrics
/latest_metrics	GET	Returns latest ROC/PR curve JSON
/reload_model	POST	Hot-reloads newest saved model

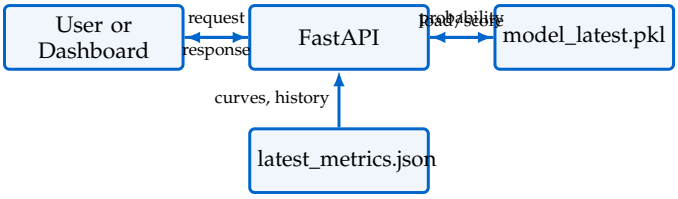

Fig. 16: Prediction serving flow in practical order.

8.2 How, Why, and When (Runbook)

TABLE 13: Operational runbook for teams

When	What to do	Why
New data arrives	Re-run Prefect pipeline	Refreshes model on latest behavior
Metrics degrade	Check PR-AUC, drift, confusion matrix	Detects quality decay early
Deployment candidate ready	Run CI + quality gate + Docker build	Prevents weak models from shipping
API starts with old model	Call reload endpoint	Activates latest model without restart

9 The Production Architecture

9.1 System Design

The system is built as a microservices architecture. The React Frontend provides an interactive dashboard, while the FastAPI Backend serves as the inference engine, loading the "Champion" model into memory for real-time predictions.

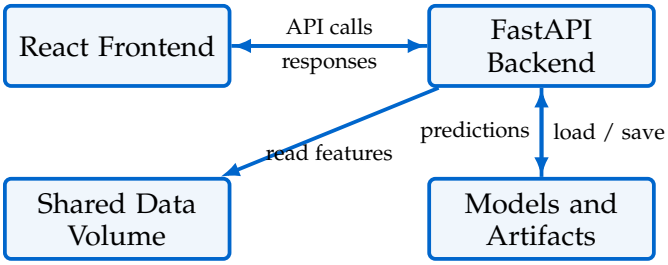

Fig. 17: Integrated system architecture showing frontend, backend, and shared volumes.

9.2 Continuous Integration and Developer Experience

We utilize a multi-stage Docker build to ensure environment parity across developer machines and production runners. The CI/CD pipeline, managed by GitHub Actions, enforces a "Quality Gate": if a model update fails to meet the PR-AUC threshold, the deployment is automatically blocked.

9.2.1 Automated Smoke Testing

To ensure the pipeline remains functional across versions, we implemented an automated "Smoke Test" in GitHub Actions. When code is pushed, the runner executes a lightweight version of the full Prefect pipeline using a sampled subset of the `dev.csv` dataset. This validates the end-to-end integration of data loading, transformation, and training without the computational overhead of the full dataset. Generated plots and models are preserved as GitHub artifacts for immediate inspection.

9.2.2 Local Developer Automation (Git Hooks)

To bridge the gap between local development and centralized orchestration, we implemented a `post-commit` Git Hook. This local script automatically triggers the Prefect pipeline on the developer's machine immediately after a commit is finalized. By setting the `PREFECT_API_URL` to the local Dockerized server, we ensure that every code change is instantly reflected in the dashboard, providing immediate feedback on model performance impacts.

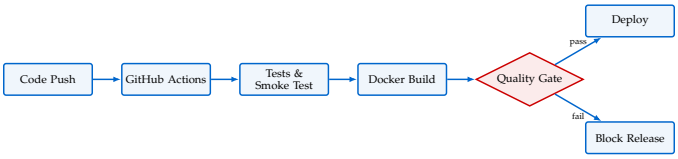

Fig. 18: Automated CI/CD workflow featuring cloud smoke testing and local Git hook triggers.

9.3 Prefect Orchestration

All machine learning tasks are orchestrated by Prefect. This ensures that data loading, training, and evaluation occur in a strict, observable sequence with automated retries and failure logging.

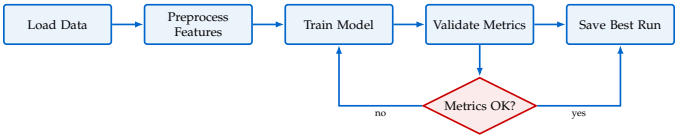

Fig. 19: Prefect orchestration flow for model training and validation.

10 Operational Notes and Future Work

10.1 How a Full Run Happens

A full run starts by loading the CSV files, then the pipeline performs EDA on the raw data, trains the model, runs EDA again on the transformed data, evaluates the result, writes metrics to disk, and only then saves the model. That order is deliberate: it gives us visibility into both the starting data and the transformed features.

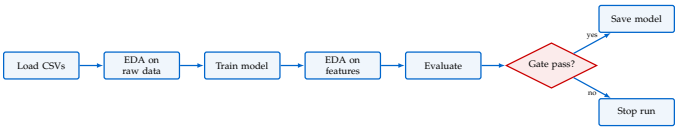

Fig. 20: The end-to-end run order used by Prefect and the training pipeline.

10.2 Monitoring and Model Health

Beyond performance metrics, the pipeline incorporates automated drift detection using the **Evidently.ai** framework. This stage generates a comprehensive "Model Health Report"

comparing the current training distribution against a historical baseline. It evaluates regression metrics, feature drift, and clustering stability, providing an early warning system for concept drift—a common failure mode in fraud detection where adversary behavior shifts over time.

TABLE 14: Project components and their operational roles

Component	Simple role	When it matters
Prefect	Orchestrates task sequencing and error handling	During training and retraining
FastAPI	Serves low-latency predictions via HTTP	Real-time inference and UI interaction
Evidently	Automated drift and health reporting	Post-training validation and monitoring
Docker	Environment parity and microservice isolation	Deployment and CI/CD validation
Git Hooks	Local automation and instant feedback	Developer workflow and iteration

10.3 Future Work

The next natural improvements are to add live drift monitoring, more robust probability calibration, and better integration between the dashboard and the latest model artifacts. Another useful step would be to add a richer explanation layer so an analyst can see which inputs pushed a prediction toward fraud.

10.4 Final Observations and Future Work

The project bridges research and production engineering by combining a measured modeling strategy with a fully operational serving stack. In practical terms, it shows that a carefully controlled tabular-learning pipeline can still perform strongly in a domain where the minority class is rare, feature redundancy is high, and the deployment environment must remain deterministic. The achieved ROC-AUC exceeds the paper benchmark, while the PR-AUC remains close to the target, confirming that the system is competitive rather than merely functional.

Future work will focus on:

- Integrating **Real-Time Feature Stores** to reduce latency in user behavioral aggregates.
- Implementing **Graph Neural Networks (GNNs)** to capture network-level fraudulent relationships.
- Expanding the dashboard to include real-time drift monitoring and automated model retraining.